YallaFix API Documentation

Smart Campus Facility Management System

Version 1.0 | May 2026

1. Overview

Base URL: http://[SERVER_IP]:3000/api

All protected routes require: **Authorization: Bearer <token>**

Content-Type: **application/json** (except multipart/form-data for file uploads)

1.1 Authentication

The API uses JWT (JSON Web Token) for stateless authentication. All protected endpoints require a valid JWT token in the Authorization header.

2. Authentication Endpoints

2.1 POST /auth/register

Description: Register a new user account

Request Body:

- name (string, required): User's full name
- email (string, required): Unique email address
- password (string, required): Password (minimum 6 characters)
- role (string, required): One of 'reporter', 'manager', 'worker'

Response 201:

- user: User object with id, name, email, role
- token: JWT token for subsequent requests

2.2 POST /auth/login

Description: Authenticate and receive a JWT token

Request Body:

- email (string, required): User's email
- password (string, required): User's password

Response 200:

- user: User object with id, name, email, role
- token: JWT token

2.3 POST /auth/logout

Description: Invalidate the current token

Requires: Valid JWT token in Authorization header

Response 200: { message: 'Logged out successfully' }

2.4 POST /auth/forgotPassword

Description: Send password reset email

Request Body:

- email (string, required): User's email

Response 200: { message: 'Password reset email sent' }

3. Complaints Endpoints

3.1 POST /complaints

Description: Submit a new complaint

Requires: **reporter** role

Content-Type: **multipart/form-data**

Request Fields:

- description (string, required): Issue description
- category_id (integer, required): Category ID
- location_id (integer, required): Location ID
- photo (file, required): Complaint image

Response 201:

- complaint: Complaint object with id, status, created_at

3.2 GET /complaints

Description: Get all complaints (manager/admin only)

Requires: **manager** or **admin** role

Query Parameters (optional):

- status: Filter by status (open, in_progress, resolved, closed)
- category_id: Filter by category
- page: Pagination (default: 1)

Response 200:

- complaints: Array of complaint objects
- total: Total complaint count

3.3 GET /complaints/my

Description: Get complaints submitted by logged-in user

Requires: Valid JWT token

Response 200:

- complaints: Array of user's complaints

3.4 GET /complaints/assigned

Description: Get complaints assigned to logged-in worker

Requires: **worker** role

Response 200:

- complaints: Array of assigned complaints

3.5 GET /complaints/:id

Description: Get full details of a specific complaint

Requires: Valid JWT token

Response 200:

- complaint: Full complaint object with metadata
- comments: Array of comments on the complaint

3.6 PUT /complaints/:id/status

Description: Update complaint status

Requires: **manager** or **worker** role

Request Body:

- status (string, required): One of 'open', 'in_progress', 'resolved', 'closed'

Response 200:

- complaint: Updated complaint object

3.7 PUT /complaints/:id/assign

Description: Assign complaint to a worker

Requires: **manager** role

Request Body:

- worker_id (integer, required): Worker's user ID

Response 200:

- complaint: Updated complaint with worker assignment

3.8 POST /complaints/:id/comments

Description: Add a comment or note to a complaint

Requires: Valid JWT token

Request Body:

- text (string, required): Comment text
- is_official (boolean, optional): Mark as official manager update

Response 201:

- comment: Created comment object

3.9 POST /complaints/:id/confirm

Description: Confirm/upvote an issue

Requires: Valid JWT token

Response 200:

- confirmation_count: Updated confirmation count
- confirmed: True if confirmation was successful

3.10 POST /complaints/:id/proof

Description: Upload proof of completion photo

Requires: **worker** role

Content-Type: **multipart/form-data**

Request Fields:

- photo (file, required): Completion proof image

Response 200:

- complaint: Updated complaint with proof_image_url

4. Notifications Endpoints

4.1 GET /notifications

Description: Get all notifications for logged-in user

Requires: Valid JWT token

Query Parameters (optional):

- unread_only: Filter to unread notifications only

Response 200:

- notifications: Array of notification objects

4.2 PUT /notifications/:id/read

Description: Mark a notification as read

Requires: Valid JWT token

Response 200:

- notification: Updated notification object with is_read=true

4.3 PUT /notifications/read-all

Description: Mark all notifications as read for logged-in user

Requires: Valid JWT token

Response 200:

- message: Confirmation message

5. Error Responses

All errors follow a standard format:

- { error: 'Error message describing the issue' }

Status	Meaning
400	Bad Request - Missing or invalid fields
401	Unauthorized - Missing or invalid JWT token
403	Forbidden - Insufficient permissions for this action
404	Not Found - Resource does not exist
409	Conflict - Resource already exists (e.g., email taken)
500	Internal Server Error - Unexpected server error

6. Security

6.1 Rate Limiting

- Login attempts: 5 requests per minute per IP
- API calls: 100 requests per minute per user

6.2 Input Validation

- All string inputs are sanitized to prevent SQL injection
- File uploads are validated for type and size (max 5MB)
- Email validation follows RFC 5322 standards